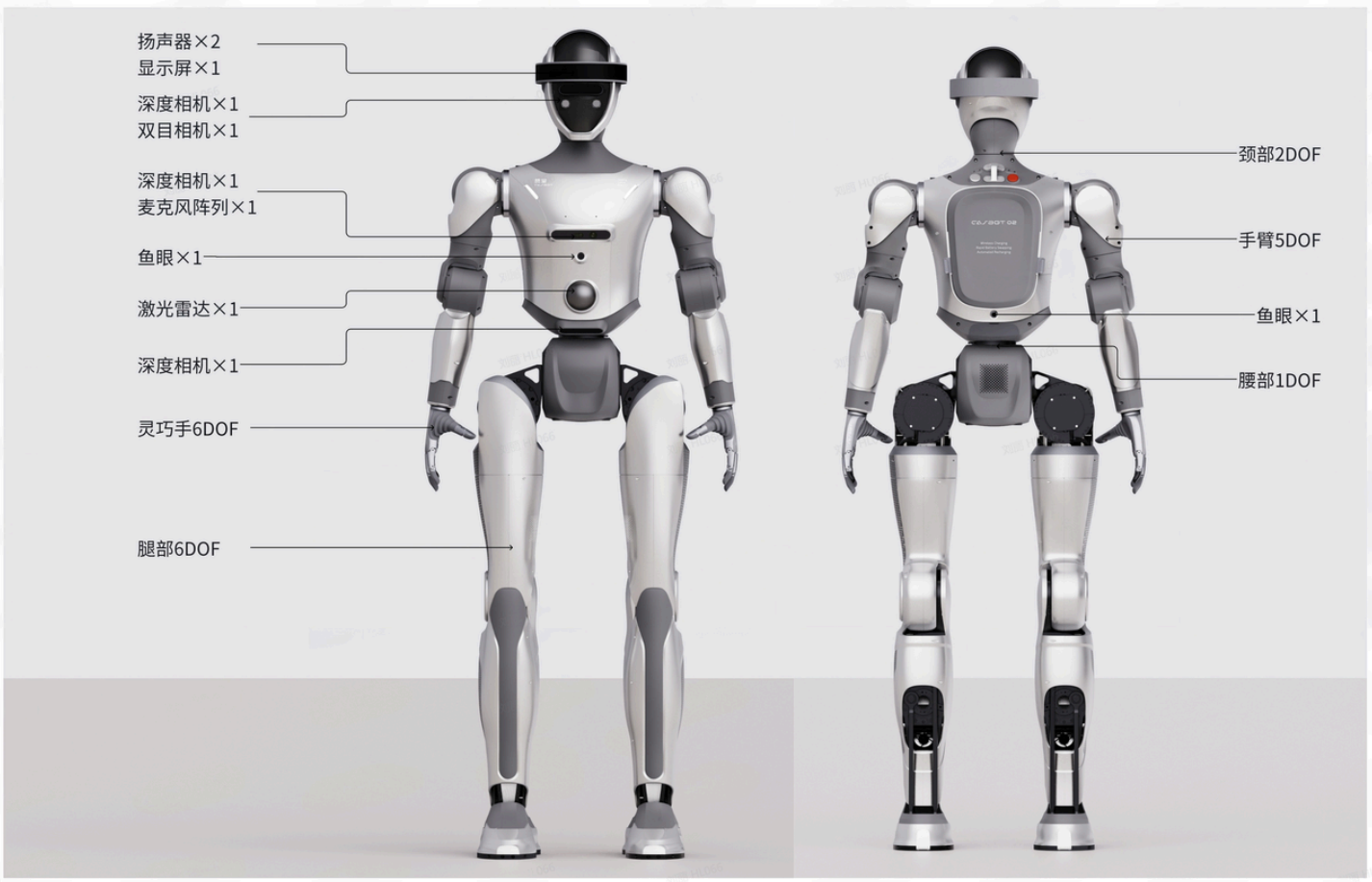
CASBOT02 二次开发文档-0305-外部小伙伴

1. 基础资料

1.1 URDF

此文档需要联系销售获取

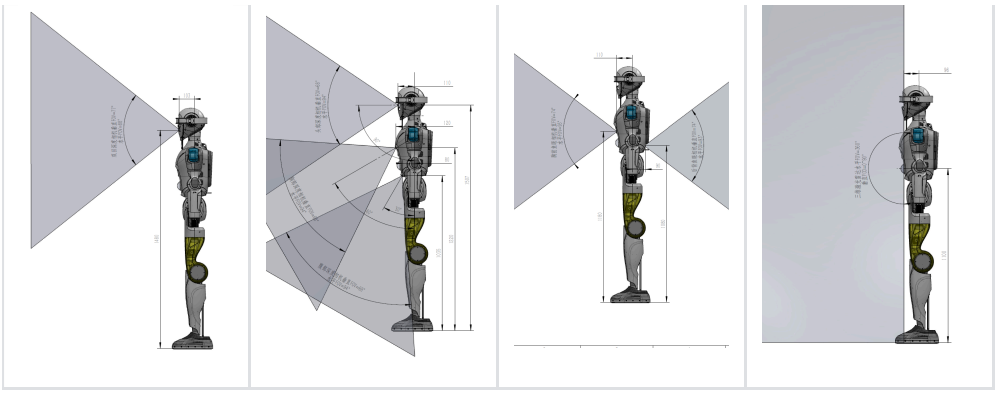
1.2 整机结构



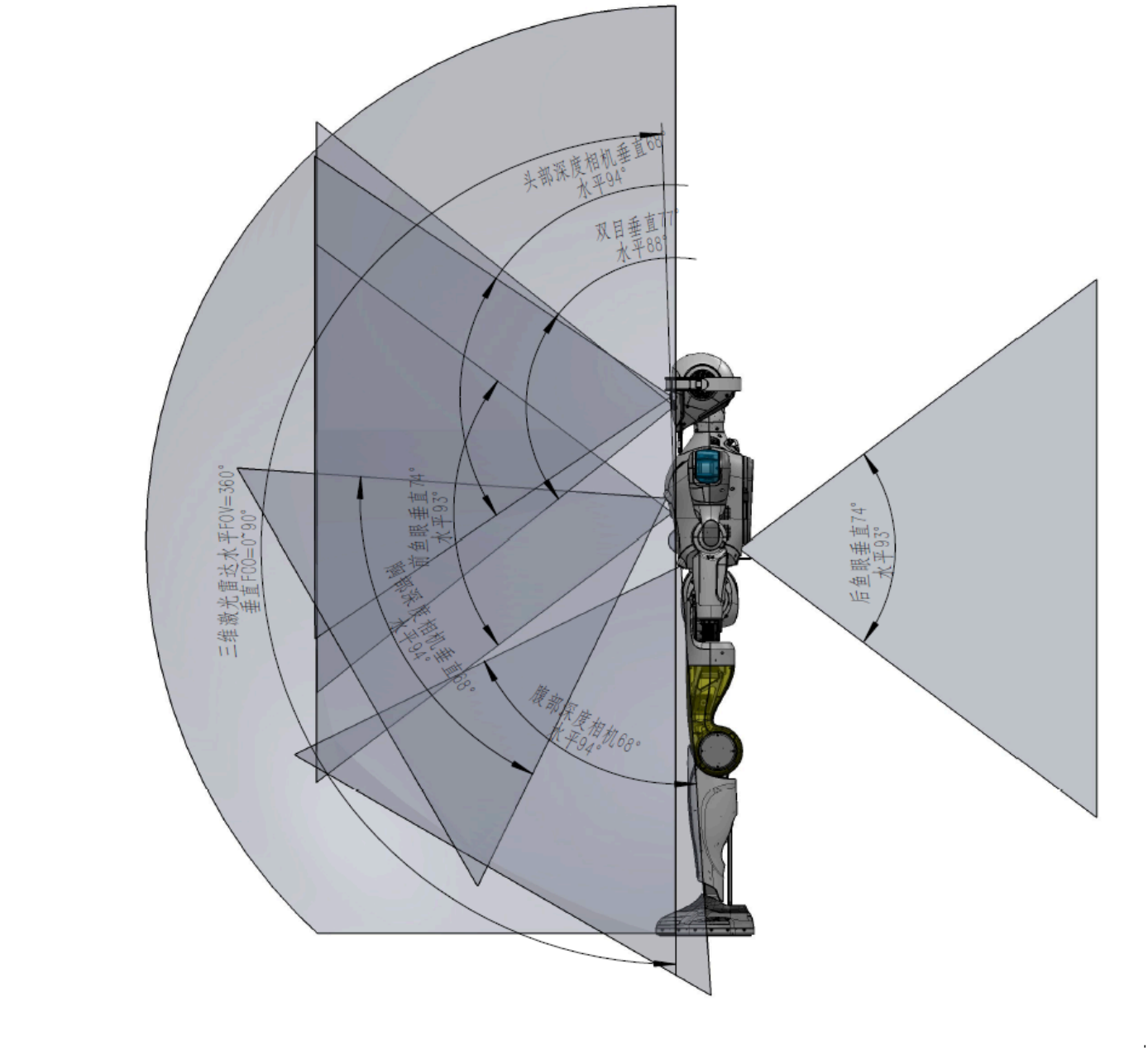
整机身高：160cm，体重：50kg

1.3 传感器感知范围

头部双目相机	深度相机	前后鱼眼相机	腹部激光雷达



总体感知范围



1.4 关节活动角度

（1）头部

关节编号	头部关节	限位（弧度）	描述	备注
1	head_yaw_joint	[-1.5708，1.5708]	头部偏航	
2	head_pitch_joint	[-0.2617993878，0.5235987756]	头部俯仰	

（2）腰部

关节编号	腰部关节	限位（弧度）	描述	备注
1	waist_yaw_joint	[-1.5708，1.5708]	腰部偏航	

（3）手臂

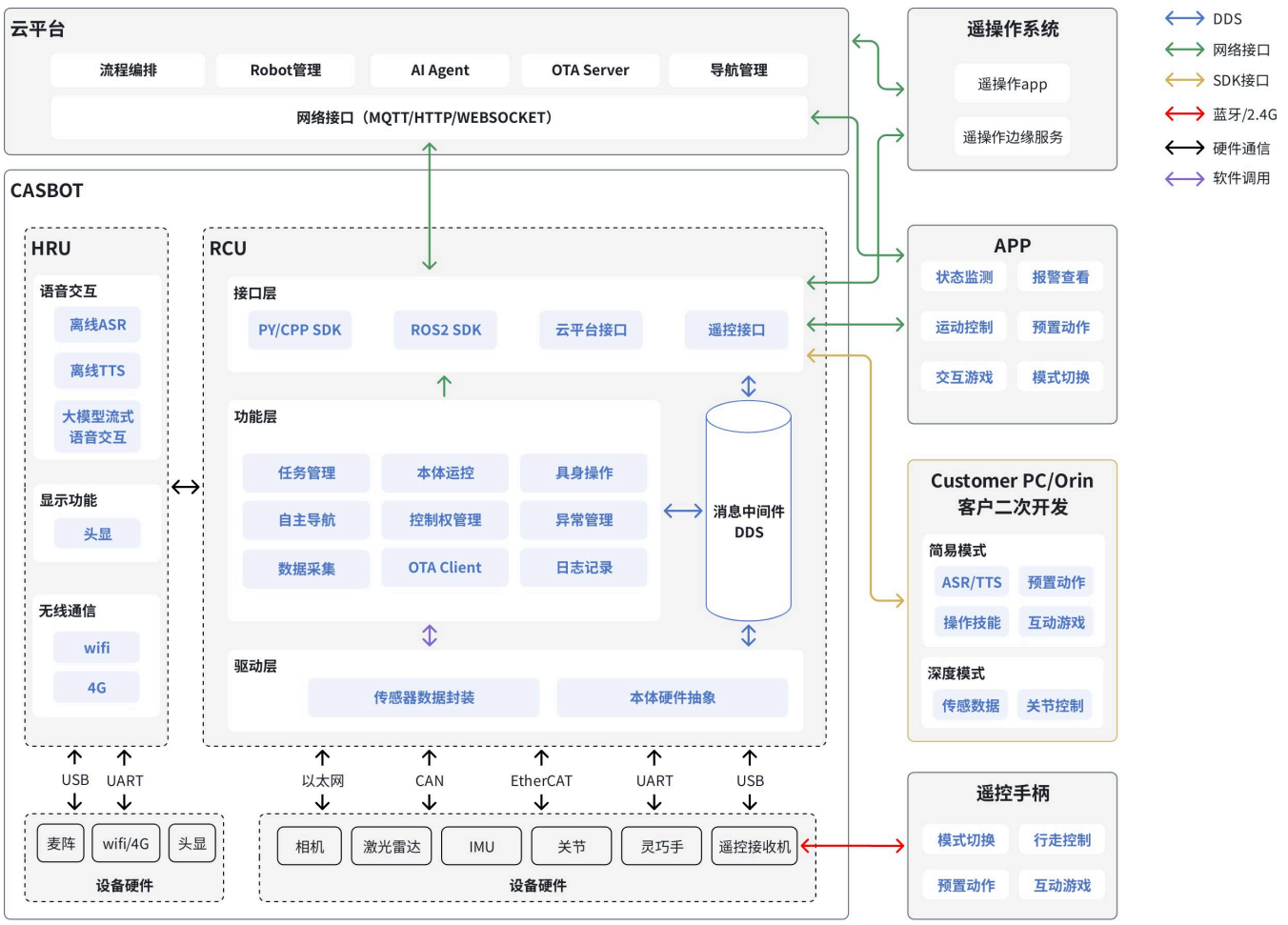
关节编号	左手臂关节	限位（弧度）	右手臂关节	限位（弧度）	描述	备注
1	left_shoulder_pitch_joint	[-3.1415926536，1.04719755]	right_shoulder_pitch_joint	[-3.1415926536，1.04719755]	肩膀俯仰	
2	left_shoulder_roll_joint	[-0.3491，3.1415926536]	right_shoulder_roll_joint	[-3.1415926536，0.3491]	肩膀滚转	
3	left_shoulder_yaw_joint	[-1.5708，1.5708]	right_shoulder_yaw_joint	[-1.5708，1.5708]	肩膀偏航	
4	left_elbow_pitch_joint	[-1.8675，0]	right_elbow_pitch_joint	[-1.8675，0]	手肘俯仰	
5	left_wrist_yaw_joint	[-1.5708，1.5708]	right_wrist_yaw_joint	[-1.5708，1.5708]	手腕偏航	
6	left_wrist_roll_joint	[-1.0472，1.5708]	right_wrist_roll_joint	[-1.5708，1.0472]	手腕滚转	选装
7		[-1.0472，1.0472]				选装

	left_wrist_pitch_joint		right_wrist_pitch_joint	[-1.0472，1.0472]	手腕俯仰	

（4）腿

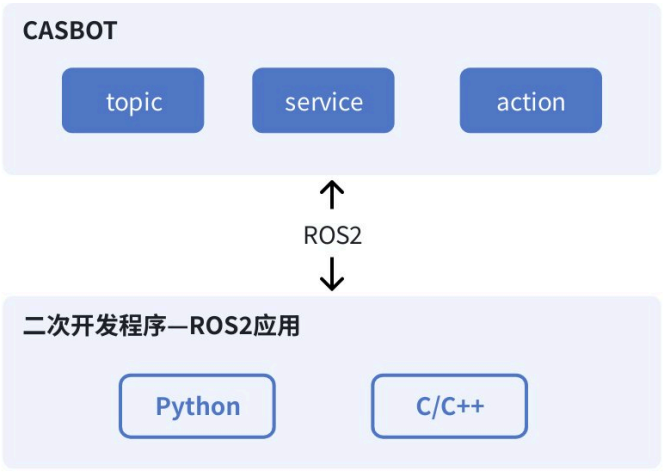
关节编号	左腿关节	限位（弧度）	右腿关节	限位（弧度）	描述	备注
1	left_leg_pelvic_pitch_joint	[-1.91986，1.5708]	right_leg_pelvic_pitch_joint	[-1.91986，1.5708]	髋部旋转	
2	left_leg_pelvic_roll_joint	[-0.17453，1.5708]	right_leg_pelvic_roll_joint	[-1.5708，0.17453]	髋部滚转	
3	left_leg_pelvic_yaw_joint	[-1.5708，1.5708]	right_leg_pelvic_yaw_joint	[-1.5708，1.5708]	髋部偏航	
4	left_leg_knee_pitch_joint	[0，2.53073]	right_leg_knee_pitch_joint	[0，2.53073]	膝盖俯仰	
5	left_leg_ankle_pitch_joint	[-0.87266，0.50614]	right_leg_ankle_pitch_joint	[-0.87266，0.50614]	脚踝俯仰	
6	left_leg_ankle_roll_joint	[-0.50614，0.50614]	right_leg_ankle_roll_joint	[-0.50614，0.50614]	脚踝滚转	

1.5 系统架构图



2. 应用开发

2.1 SDK说明



CASBOT02采用ROS2的通信机制，二次开发的SDK接口形式也为ROS2的topic/service/action三种类型。

- Topic

CASBOT02 ——> 二次开发程序：关节状态、imu数据、rgbd相机数据等；

二次开发程序 ——> CASBOT02：关节控制数据、遥控器数据；

- Service

CASBOT02 Service Server：执行技能行为树、语音对话开启关闭、状态切换；

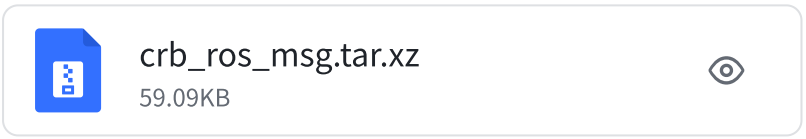
- Action

CASBOT02 Action Server：语音播报；

2.2 语音对话

- **实时对话功能开启/关闭**

调用方式为 ros2 service(服务)：
service name：voice_svr
service type：Voice.srv
依赖包：



代码块

```
开启实时对话
ros2 service call /voice_svr crb_ros_msg/srv/Voice "{type: 'rtc_start', content_type: '', content: ''}"
关闭实时对话
ros2 service call /voice_svr crb_ros_msg/srv/Voice "{type: 'rtc_stop', content_type: '', content: ''}"
提问
ros2 service call /voice_svr crb_ros_msg/srv/Voice "{type: 'question', content_type: 'text', content: '你好，你叫什么名字'}"
回答
ros2 service call /voice_svr crb_ros_msg/srv/Voice "{type: 'answer', content_type: 'object_string', content: '我叫CASBOT02,很高兴见到你'}"
```

- **音频播放**

调用方式为ros2 action(动作)：

action name：action_voice_play
action type ： VoicePlay.action
依赖包：crb_ros_msg

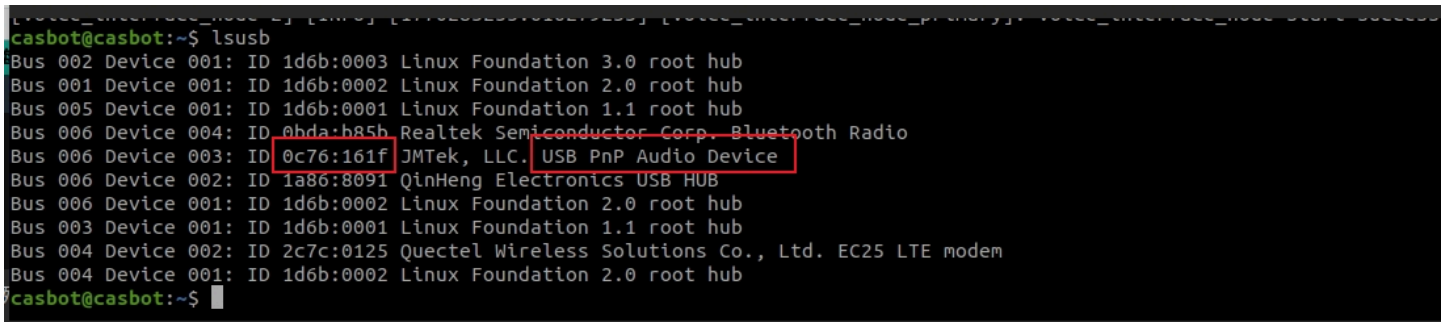
```
音频文件存放在hru的/workspace/prod_hru/share/voice_interface/resource/voice_files
目录
音频格式：
Endcoding:Signed 16-bit
Channels:单声道
sample rate:16000
format：wav

调用示例：
ros2 action send_goal /action_voice_play crb_ros_msg/action/VoicePlay
'{"wav_path":"音频名称.wav"}'

```

- **MIC/Speaker硬件描述**

CASBOT02的MIC和Speaker都是连接在HRU上，喇叭和麦克风是一体的，使用lsusb命令查看设备信息如下图所示。



2.3 预设技能

用于远程控制机器人执行预设技能或自定义动作、指令

预设技能的调用方式为ros2 service(服务)：
service name：/casbot/event_service
service type：ActionEvent.srv

- 自定义依赖包：

crb_ros_msg.tar.xz
137.37KB

```
调用示例：
ros2 service call /casbot/event_service crb_ros_msg/srv/ActionEvent
'{"event_id":"","event_type":"ExecSkill","blocking":false,"param_json": "
{\"payload\": \"{\\\"action_type\\\":\\\"wave_hand\\\"}\", \"target_tree\":
\"basic_action_play\"}"}'
```

- 预设技能清单：

技能名称	target_tree	payload
点赞	basic_action_play	"{\"action_type\"：\"thumb_up\"}"
挥手	basic_action_play	"{\"action_type\"：\"wave_hand\"}"
比心	basic_action_play	"{\"action_type\"：\"heart_gesture\"}"
恭喜	basic_action_play	"{\"action_type\"：\"congratulation_gesture\"}"
剪刀手/耶	basic_action_play	"{\"action_type\"：\"v_gesture\"}"
握手	hand_shake	"{}"
自我介绍	self_introduction	"{}"
石头剪刀布	rock_paper_scissors	"{\"robot_gesture\"：\"rock\"}" //rock:石头，scissors:剪刀，paper：布

2.4 技能定制

目前支持的预设技能和定制技能都包含**动作**和**音频**两个并行播放的元素，例如点赞技能的两个元素为thumb_up.data、thumb_up.wav。

- 动作data文件格式：

共61列数据，数据之间以逗号分隔，数据帧率为50HZ，旧版程序相邻帧之间无插值，因此相邻帧手臂末端笛卡尔空间距离不能过大。

每行数据具体内容为：Base数据（1-12）、左腿1-6关节角度（13-18）、右腿1-6关节角度（19-24）、左臂1-7关节角度（25-31）、右臂1-7关节角度（32-38）、左手1-10手指关节角度（39-48）、右手1-10手指关节角度（49-58）、头部1-2关节角度（59-60）、腰部关节角度（61）。具体内容详情见下表。

.Data数据组解析	1-12列	13-18列（左腿1-6关节）	19-24 列（右腿1-6关节）	25-31列（左臂1-7关节）	32-38列（右臂1-7关节）	39-48列（左臂1-10手指）	49-58列（右臂1-10手指）	59-60列（头部1-2关节）	61列（腰部）
顺序1	Base_向X方	left_leg_pelvic	right_leg_pelvic_	left_shoulder_pi	right_shoulder_	left_thumb_metacarpal_joint	right_thumb_metacarpal_joint	head_yaw_joint	waist_yaw_joint

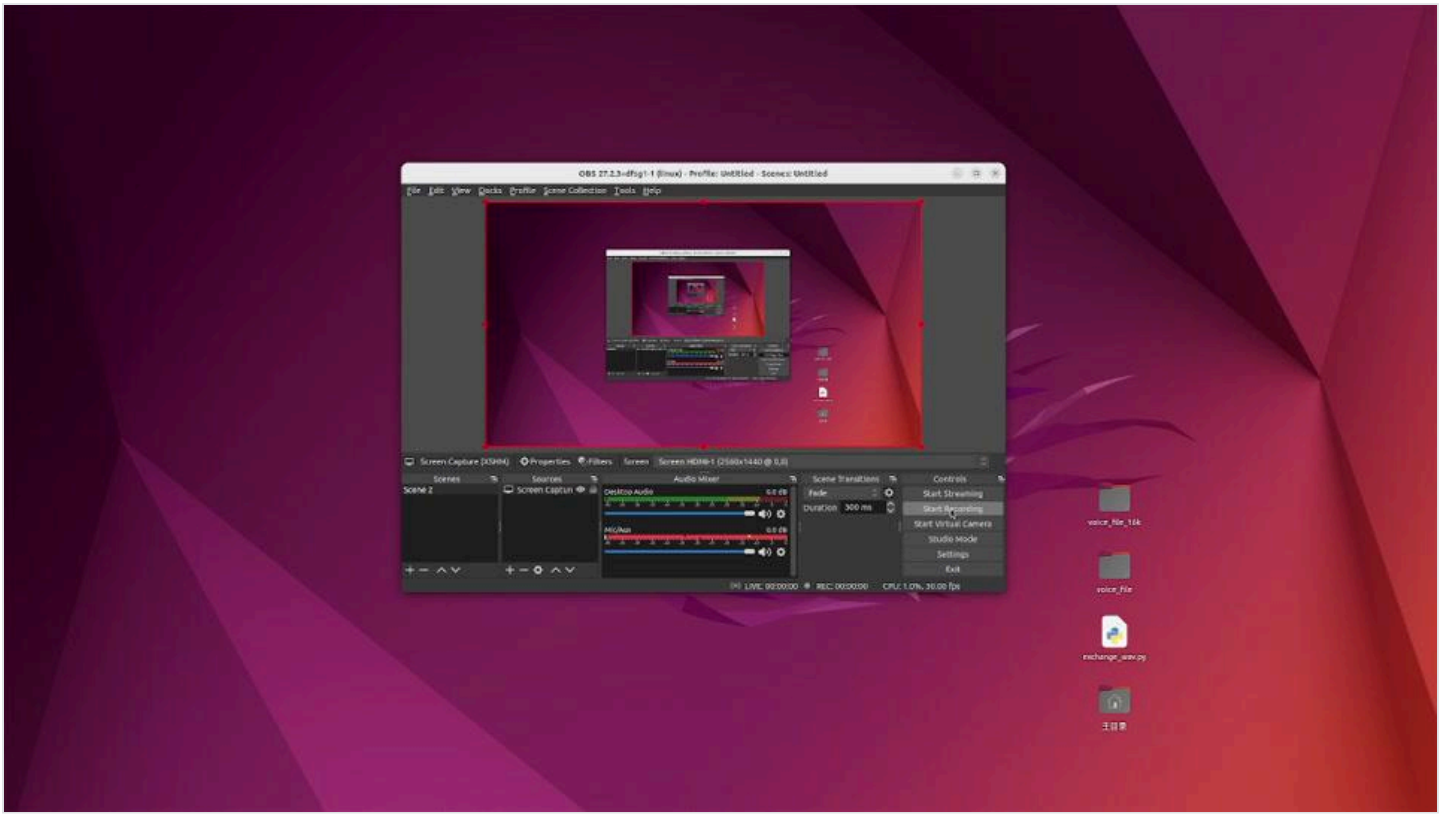
	向平移（机器人左右横移）	_pitch_joint	pitch_joint	tch_joint	pitch_joint				
顺序2	Base_向Y方向平移(机器人面部朝向-前后运动）	left_leg_pelvic_roll_joint	right_leg_pelvic_roll_joint	left_shoulder_roll_joint	right_shoulder_roll_joint	left_thumb_proximal_joint	right_thumb_proximal_joint	head_pitch_joint	
顺序3	Base_向Z方向平移（机器人上下运动）	left_leg_pelvic_yaw_joint	right_leg_pelvic_yaw_joint	left_shoulder_yaw_joint	right_shoulder_yaw_joint	left_index_proximal_joint	right_index_proximal_joint		
顺序4	Base_Pitch方向旋转（机器人前趴后仰）	left_leg_knee_pitch_joint	right_leg_knee_pitch_joint	left_elbow_pitch_joint	right_elbow_pitch_joint	left_index_distal_joint（被动）	right_index_distal_joint（被动）		
顺序5	Base_Yaw方向旋转（机	left_leg_ankle_pitch_joint	right_leg_ankle_pitch_joint	left_wrist_yaw_joint	right_wrist_yaw_joint	left_middle_proximal_joint	right_middle_proximal_joint		

	器人左右转身）								
顺序6	Base_Roll方向旋转（机器人左倾右倾）	left_leg_ankle_roll_joint	right_leg_ankle_roll_joint	left_wrist_pitch_joint	right_wrist_pitch_joint	left_middle_distal_joint（被动）	right_middle_distal_joint（被动）		
顺序7	Base_X方向线速度			left_wrist_roll_joint	right_wrist_roll_joint	left_ring_proximal_joint	right_ring_proximal_joint		
顺序8	Base_Y方向线速度					left_ring_distal_joint（被动）	right_ring_distal_joint（被动）		
顺序9	Base_Z方向线速度					left_pinky_proximal_joint	right_pinky_proximal_joint		
顺序10	Base_Pitch角速度					left_pinky_distal_joint（被动）	right_pinky_distal_joint（被动）		
顺序11	ase_Yaw角速度								
顺序12	Base_Roll角速度								

- 音频wav文件格式

要求音频为单声道，采样率为16000。

1.浏览器登录扣子平台：www.coze.cn
2.获取个人访问令牌
3.语音合成下载音频
4.拷贝音频到hru指定目录下
5.ros2 action调用播放



- 技能定制流程

（1）基于动画设计软件或Rviz生成CASBOT02的上身运动轨迹数据，参照上面的data文件格式保存为test.data，并保存到Orin的/workspace/prod_casbot02_basic/share/crb_resources/resources/motion/目录下。

(2) 将需要播放的音频存为test.wav，并保存到HRU的/workspace/prod_hru/share/crb_resources/resources/voice_files_cn或 /workspace/prod_hru/share/crb_resources/resources/voice_files_en目录下，区分中英文。

（3）当机器人处于准备模式或运动模式时，执行以下命令即可播放定制的动作和音频

代码块

```
调用示例:
ros2 service call /casbot/event_service crb_ros_msg/srv/ActionEvent
'{"event_id":"","event_type":"ExecSkill","blocking":false,"param_json": "
{\"payload\": \"{\\\"action_type\\\":\\\"wave_hand\\\"}\", \"target_tree\":
\"basic_action_play\"}"}'
```

2.5 传感器数据

2.5.1 相机数据

casbot02本体上相机毕竟多，在头部/胸口/腹部共有3个RGBD相机，在头部还有一个彩色双目相机。相机的数据获取都采用ros2 topic的方式，具体定义如下表：

接口名称/功能描述	接口参数(ROS2 topic)	返回值	说明
视觉传感器数据 • 彩色图	• /camera_xxx/color/image_raw • /camera_xxx/color/image_raw/compressed • RGBD 相机有 /camera_xxx/color/camera_info	• Type: sensor_msgs/msg/Image • 压缩图 Type: sensor_msgs/msg/CompressedImage Size：1280*720	camera_xxx 相机名称： RGBD相机有 camera_head | camera_chest | camera_belly 双目彩色相机有 camera_stereo 鱼眼相机有 camera_fisheyes_front | camera_fisheyes_back
视觉传感器数据 • 彩色图小图（彩色图一半长宽）	• /camera_xxx/color/image_raw_mini • /camera_xxx/color/image_raw_mini/compressed	• Type: 同上 Size：640*320	RGBD相机、双目彩色相机、鱼眼相机，名称同上
视觉传感器数据 • 深度图	• /camera_xxx/depth/image_raw • /camera_xxx/depth/image_raw/compressed • RGBD 相机有 /camera_xxx/depth/camera_info	• 同上 Size：1280*720	RGBD相机，名称同上
视觉传感器数据 • 深度图小图（深度图一半长宽）	• /camera_xxx/depth/image_raw_mini • /camera_xxx/depth/image_raw_mini/compressed	• 同上 Size：640*320	RGBD相机，名称同上

2.5.2 IMU数据

IMU数据采用ros2系统自带的topic，具体说明如下：

接口名称/功能描述	接口参数(ROS2 topic)	返回值	说明
惯性传感器数据	• /imu	`Type: sensor_msgs/msg/Imu`	用于机器人姿态估算、运动状态分析（如平衡控制）

2.5.3 关节状态数据

关节数据采用ros2系统自带的topic，具体说明如下：

接口名称/功能描述	接口参数(ROS2 topic)	返回值	说明
关节电驱信息 位置、速度、力矩	• /joint_states	Type: sensor_msgs/msg/JointState	包含腿部、手臂、、腰部、头部、灵巧手关节

2.5.4 激光雷达数据

CASBOT02采用的激光雷达是速腾聚创Ariy，且激光雷达是单独接在导航控制模块上的。由于导航控制模块与Orin挂接在同一个交换机上，在同一个局域网，二次开发程序也能访问到激光雷达数据，具体topic定义如下表所示：

接口名称/功能描述	接口参数(ROS2 topic)	返回值	说明
激光雷达原始点云数据	/rslidar_points	Type: sensor_msgs/msg/PointCloud2	Airy雷达原始点云数据

如有些客户需要运行自己的导航算法，要求激光雷达直接连接到Orin，需要从Orin读取激光雷达数据。则需在Orin上参照github（https://github.com/RoboSense-LiDAR/rslidar_sdk/?tab=readme-ov-file）项目安装激光雷达驱动。

2.5.4.1 激光雷达数据

CASBOT02采用的激光雷达是速腾聚创Airy，且激光雷达直接与Orin挂接在同一个网口下（不经过导航控制模块），处于同一个局域网内。二次开发程序能够直接访问到激光雷达的点云及惯导数据，具体topic定义如下表所示：

接口名称/功能描述	接口参数(ROS2 topic)	返回值	说明
激光雷达原始点云数据	/casbot/rslidar_points	Type: sensor_msgs/msg/PointCloud2	Airy雷达原始点云数据
激光雷达IMU惯导数据	/casbot/rslidar_imu_data	Type: sensor_msgs/msg/Imu	Airy雷达内置IMU惯导原始数据

2.6 头显通讯协议

2.6.1 本协议基于 **UART+RS485 总线** 设计，用于主设备向从设备发送显示控制指令，支持两种显示模式：ASCII 字符显示、预设图片显示

2.6.2通信机制

- 通信模式：主从模式（主设备发送指令，从设备响应并执行显示操作）
- 响应逻辑：从设备收到指令后，解析正确后进行总线响应，否则无响应
- 数据格式：字节流传输，无校验位
- 指令有效条件：接收数据长度 2 < len < 256（避免数据溢出或无效帧）

2.6.3数据帧整体结构

所有指令帧遵循统一的头部格式，数据段随显示模式变化：

字节偏移	字段名称	长度（字节）	含义说明
0	设备 ID	1	从设备地址，必须匹配 DEVICE_ID（0x01） 才解析
1	显示模式	1	枚举值：0=ASCII 模式，1 = 图片模式，4 = 投射模式
2~n-1	模式数据段	可变	不同显示模式的具体参数 + 数据（见下文）

2.6.4各模式详细协议说明

2.6.4.1Mode_Ascii（ASCII 字符显示模式）

数据帧结构

字节偏移	字段名称	长度（字节）	含义说明
0	设备 ID	1	匹配 DEVICE_ID, 0x01
1	显示模式	1	固定为 0（Mode_Ascii）
2~4	颜色参数	3	red (2)、green (3)、blue (4)，取值 0~0xFF
5	水平偏移	1	offset_x：字符显示起始 X 坐标（左上角为原点）
6~n-1	ASCII 字符串	可变	以 '\0' 结尾的 ASCII 字符串（32~126）

依赖结构体

代码块

```
//1字节对齐
typedef struct {
   uint8_t dev_id;     // 设备DI，0x00
    uint8_t disp_mode; // 显示模式，0x01
    uint8_t red;        // 字符颜色-红通道（0~0xFF）
    uint8_t green;      // 字符颜色-绿通道（0~0xFF）
    uint8_t blue;       // 字符颜色-蓝通道（0~0xFF）
    uint8_t offset_x;  // 起始X坐标（左下角为原点）
    char str[];         // 可变长ASCII字符串（'\0'结尾）
} s_Mode0Frame_t
```

功能说明：

- 支持显示标准 ASCII 字符（32~112，即空格～波浪号）
- 字符点阵规格：8x13（宽 8 像素，高 13 像素）
- 字符间距：无额外间距，连续显示时自动右移 8 像素
- 超出屏幕范围的字符自动截断（不显示）

2.6.4.2Mode_Picture（预设图片显示模式）

数据帧结构

字节偏移	字段名称	长度（字节）	含义说明
0	设备 ID	1	匹配 DEVICE_ID
1	显示模式	1	固定为 1（Mode_Picture）
2	图片索引	1	0 = 清屏，1~BMP_NUM = 预设图片编号
3~5	颜色参数	3	rev[0]=red、rev[1]=green、rev[2]=blue
6~n-1	保留字段	0	无额外数据，图片数据通过索引从本地获取

数据结构体

代码块

```
//1字节对齐
typedef struct {
    uint8_t dev_id;     // 设备id，  0x01
    uint8_t disp_mode; // 显示模式，0x02
    uint8_t index;      // 图片索引（0=清屏，1~BMP_NUM）
    uint8_t rev[3];     // 图片显示颜色（red/green/blue）
} s_Mode1or2Frame_t;
```

功能说明

- 图片数据存储：预存图片在MCU FLASH中，格式为单色位图
- 图片规格：1 位单色位图，尺寸固定为屏幕尺寸（112 x 13）
- 显示位置：固定显示在屏幕左上角（X=0，Y=0）
- 清屏指令：当 `index=0` 时，执行清空屏幕所有像素

预设图片枚举（图片尺寸112x13）

Index	字段名称	图片
0x01	casbot 02	
0x0D	Zzz	
0x0E	WOW	
0x0F	welecome	
0x10	hey bro	
0x11	bravo	
0x12	smile	
0x13	yeh	
0x14	game	

2.6.4.3Mode_Casting（多区域投射显示模式）

数据帧结构

字节偏移	字段名称	长度（字节）	含义说明
0	设备 ID	1	匹配 DEVICE_ID
1	显示模式	1	固定为 2（Mode_Casting）
2	图片索引	1	0 = 清屏，1~BMP_NUM = 无效（仅清屏有效）
3	投射命令个数	1	cmd_num：单帧内包含的投射指令个数（≤MAX_CASTING_CMD_NUM）
4~n-1	投射指令集	可变	cmd_num 个 s_ModeCastFrame_t 结构体（连续排列）

核心依赖结构体

（1）模式头部结构体

代码块

```
//单字节对齐
typedef struct {
    uint8_t dev_id;     // 设备ID
    uint8_t disp_mode; // 显示模式，0x03
    uint8_t index;      // 0=清屏，非0无效
```

```
    uint8_t cmd_num;    // 投射命令个数（1~9）
} s_Mode4Frame_t;
```

（2）单条投射指令结构体（紧跟指令头）

代码块

```
//单字节对齐
typedef struct {
    uint8_t len;        // 当前投射指令总长度（包含len~data及后续图像数据）
    uint8_t red;        // 位图颜色-红通道（0~0xFF）
    uint8_t green;      // 位图颜色-绿通道（0~0xFF）
    uint8_t blue;       // 位图颜色-蓝通道（0~0xFF）
    uint8_t x;          // 显示起始X坐标（左上角为原点）
    uint8_t y;          // 显示起始Y坐标（左上角为原点）
    uint8_t width;      // 位图宽度（像素，1~112）
    uint8_t heigth;     // 位图高度（像素，1~13）
    uint8_t data;       // 1位位图数据起始位置（后续字节均为位图数据）
} s_ModeCastFrame_t;
```

投射指令关键说明

1. **指令长度计算**： len = 8（结构体固定长度） + 位图数据字节数 （不含头部）= (width + 7) / 8 * heigth（1 字节 = 8 像素，向上取整）
2. **指令偏移逻辑**：多条投射指令连续存储，通过 `len` 偏移获取下一条指令：`mode_cast_frame = (s_ModeCastFrame_t *)((uint8_t *)mode_cast_frame + mode_cast_frame->len)`
3. **位图数据格式**：1 位单色位图，高位（MSB）对应左列像素，低位（LSB）对应右列像素

2.6.5 示例（结合ros2 topic）

1.显示ASCII字符串 “Welcome to Uart” ，x坐标：0， 字体颜色：0x0F0F0F (颜色)

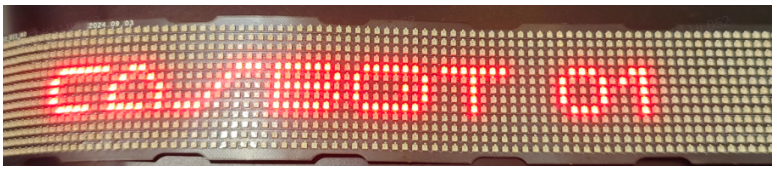
ros2 topic pub /rcu/mcu/serial_face std_msgs/msg/String "{data: '0100000F0F0F0000000057656C636F6D6520746F205561727400'}" --once

2.清屏, 模式01，图片索引00

ros2 topic pub /rcu/mcu/serial_face std_msgs/msg/String "{data: '01010000'}" --once

3.显示图片 "型号"，颜色为红色（0x0F0000）

ros2 topic pub /rcu/mcu/serial_face std_msgs/msg/String "{data: '0101010F00000000'}" --once



ros2 topic pub /rcu/mcu/serial_face std_msgs/msg/String "{data: '01010D0F00000000'}" --once



ros2 topic pub /rcu/mcu/serial_face std_msgs/msg/String "{data: '01010E0F00000000'}" --once



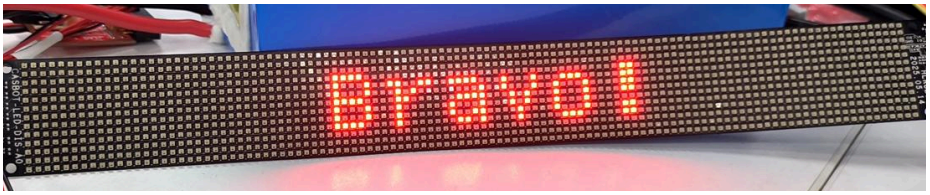
ros2 topic pub /rcu/mcu/serial_face std_msgs/msg/String "{data: '01010F0F00000000'}" --once`



ros2 topic pub /rcu/mcu/serial_face std_msgs/msg/String "{data: '0101100F00000000'}" --once



ros2 topic pub /rcu/mcu/serial_face std_msgs/msg/String "{data: '0101110F00000000'}" --once



ros2 topic pub /rcu/mcu/serial_face std_msgs/msg/String "{data: '0101120F0F0F0000'}" --once



ros2 topic pub /rcu/mcu/serial_face std_msgs/msg/String "{data: '0101130F0F0F0000'}" --once



ros2 topic pub /rcu/mcu/serial_face std_msgs/msg/String "{data: '0101140F0F0F0000'}" --once



模式3



```
01 03 01 02 0F 00 00 0F 00 00 08 07 10 20 60 FC 18 10 20  7D 0F 00 00
14 00 48 0D 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 07 E6 00 EF C7 E7
F0 39 E0 08 09 01 08 28 10 80 44 10 08 10 82 0F C8 50 80 44 E0 08 10
84 08 28 10 80 45 00 07 9E B8 0F C7 E0 80 39 F0 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
```

头部（对应模式 3 的帧头结构，共 4 字节）

字节：`01 03 01 02`

- `01`：`dev_id`（设备 ID=1，指定目标设备）；
- `03`：`disp_mode`（显示模式 = 0x03，即投影模式，符合用户说明）；
- `01`：`index`（非 0 值，不清屏，仅 0 时执行清屏）；
- `02`：`cmd_num`（共 2 条投影指令，后续紧跟 2 条 `s_ModeCastFrame_t` 结构的指令）。

第一条投影指令（共 15 字节，`len=0x0F`）

字节范围：`0F 00 00 0F 00 00 08 07 10 20 60 FC 18 10 20`

1. 固定参数部分（前 8 字节）：
   - `0F`：`len`（当前指令总长度 = 15 字节，8 字节固定部分 + 7 字节图像数据）；
   - `00 00 0F`：`red=0x00`、`green=0x00`、`blue=0x0F`（低亮度蓝色，RGB 组合）；
   - `00 00`：`x=0`、`y=0`（显示起始坐标，左上角原点 (0,0)）；
   - `08 07`：`width=8`、`height=7`（图像尺寸 8×7 像素，对应 `gImage_power`）。
2. 图像数据部分（后 7 字节）：`10 20 60 FC 18 10 20`：`gImage_power` 的 1 位位图像素数据（电源图标图案）。
3. 第二条投影指令（共 125 字节，`len=0x7D`）

字节范围：`7D 0F 00 00 14 00 48 0D ...`（后续为 117 字节图像数据）

1. 固定参数部分（前 8 字节）：
   - `7D`：`len`（当前指令总长度 = 125 字节，8 字节固定部分 + 117 字节图像数据）；
   - `0F 00 00`：`red=0x0F`、`green=0x00`、`blue=0x00`（低亮度红色，RGB 组合）；
   - `14 00`：`x=20`（0x14=20）、`y=0`（显示起始坐标 (20,0)，横向偏移 20 像素避免重叠）；
   - `48 0D`：`width=72`（0x48=72）、`height=13`（0x0D=13）（图像尺寸 72×13 像素，对应 `gImage_casbot_02`）。

2. 图像数据部分（后 117 字节）：后续字节为 `gImage_casbot_02` 的 1 位位图像素数据（72×13 像素的图形）

2.6.6 OTA

版本信息读取

发送 01 04 FF FF

返回 64Bytes 数据（32Bytes 设备名称字串，32Bytes 版本字串）

发送 01 05 FF FF，进入BOOT OTA，协议采用Xmodem，协议链接：

https://blog.csdn.net/weixin_35987118/article/details/151871708
Xmodem文件传输协议原理与应用详解-CSDN博客
文章浏览阅读1.2k次，点赞21次，收藏30次。起始标志SOH(Start of Header)是Xmodem协议中用于标识一个新数据块开始的单字节控制字符，其ASCII码值为0x01…

3. 运控开发

3.1 下肢行走控制接口

- **控制权切换**

采用手柄切换：
RB+A切换导航模式

RB+Y切换遥控模式

- **行走控制接口**

通过手柄将robot切换到导航模式后，才可以使用本控制接口，接口的topic名称定义为/navigation/cmd_vel，接口类型采用ros2系统自带的geometry_msgs/msg/Twist消息类型。

`linear.x`：前后运动速度

`linear.y`：左右平移速度

`angular.z`：绕竖直轴的旋转角速度

3.2 上身关节控制接口

（1）上身调试模式服务请求

CASBOT02需要先申请上身调试模式，才能通过接口控制上身关节。

通讯方式：ros2 service

service name: /motion/upper_body_debug

service type: std_srvs::srv::SetBool

申请进入上身调试：则设置参数为true

申请退出上身调试：则设置参数为false

（2）上身关节控制接口

上身包括头、腰、双臂、灵巧手几个部分。

- **关节名称**

身体部位	关节名称
头	head_yaw_joint 、head_pitch_joint
腰	waist_yaw_joint
左臂	left_shoulder_pitch_joint 、left_shoulder_roll_joint 、left_shoulder_yaw_joint 、left_elbow_pitch_joint 、left_wrist_yaw_joint 、left_wrist_pitch_joint 、left_wrist_roll_joint
右臂	right_shoulder_pitch_joint 、right_shoulder_roll_joint 、right_shoulder_yaw_joint 、right_elbow_pitch_joint 、right_wrist_yaw_joint 、right_wrist_pitch_joint 、right_wrist_roll_joint
左手	left_thumb_metacarpal_joint、left_thumb_proximal_joint、left_index_proximal_joint、left_middle_proximal_joint、left_ring_proximal_joint、left_pinky_proximal_joint
右手	right_thumb_metacarpal_joint、right_thumb_proximal_joint、right_index_proximal_joint、right_middle_proximal_joint、right_ring_proximal_joint、right_pinky_proximal_joint

- **接口定义**

通讯方式：ros2 service

topic name: /upper_body_debug/joint_cmd

topic type: crb_ros_msg::msg::UpperJointData

代码块

```
# UpperJointData.msg

std_msgs/Header header

# 执行时间,单位秒
float32 time_ref
```

```
7
8   # 速度比例 [0.0,1.0]
9   float32 vel_scale
10
11  sensor_msgs/JointState joint
```

可以单帧、离散多帧、连续帧发送以上topic，对上身关节进行关节空间的位置控制。

3.3 全身关节控制接口

（1）全身调试模式服务请求

通讯方式：ros2 service

CASBOT02需要先申请全身调试模式，才能通过接口控制全身关节。

service name: /motion/whole_body_debug

service type: std_srvs::srv::SetBool

申请进入全身调试：则设置参数为true

申请退出全身调试：则设置参数为false

目前也可以直接通过遥控手柄切换到调试模式，遥控组合按键为“长按LB + 单击Y”

（2）全身关节名称

身体部位	关节名称
头	head_yaw_joint 、head_pitch_joint
腰	waist_yaw_joint
左臂	left_shoulder_pitch_joint 、left_shoulder_roll_joint 、left_shoulder_yaw_joint 、left_elbow_pitch_joint 、left_wrist_yaw_joint 、left_wrist_pitch_joint 、left_wrist_roll_joint
右臂	right_shoulder_pitch_joint 、right_shoulder_roll_joint 、right_shoulder_yaw_joint 、right_elbow_pitch_joint 、right_wrist_yaw_joint 、right_wrist_pitch_joint 、right_wrist_roll_joint
左手	left_thumb_metacarpal_joint、left_thumb_proximal_joint、left_index_proximal_joint、left_middle_proximal_joint、left_ring_proximal_joint、left_pinky_proximal_joint
右手	right_thumb_metacarpal_joint、right_thumb_proximal_joint、right_index_proximal_joint、right_middle_proximal_joint、right_ring_proximal_joint、right_pinky_proximal_joint
左腿	

	left_leg_pelvic_pitch_joint、left_leg_pelvic_roll_joint、left_leg_pelvic_yaw_joint、 left_leg_knee_pitch_joint、left_leg_ankle_pitch_joint、left_leg_ankle_roll_joint
右腿	right_leg_pelvic_pitch_joint、right_leg_pelvic_roll_joint、right_leg_pelvic_yaw_joint、 right_leg_knee_pitch_joint、right_leg_ankle_pitch_joint、right_leg_ankle_roll_joint

（3）全身关节控制接口

- **获取robot运动模式**

话题名称	通信类型	数据格式	说
/motion/current_mode	Publisher	std_msgs/msg/Int32	robot

robot模式返回值：

阻尼模式——1，准备模式——2，全身调试模式——7，上半身调试模式——8

- **获取运控状态数据**

话题名称	通信类型	数据格式	说
/motion/joint_state	Publisher	sensor_msgs::msg::JointState	关节
/motion/imu	Publisher	sensor_msgs::msg::Imu	IMU

- **获取遥控器摇杆数据**

话题名称	通信类型	数据格式	说
/motion/cmd_vel	Publisher	geometry_msgs/msg/Twist	遥杆数

二开程序可以订阅该话题数据，用于控制robot行走。

- **关节控制接口**

话题名称	通信类型	数据格式	说
/motion/joint_cmd	Subscriper	sensor_msgs::msg::JointState	关节

3.4 强化学习运控开发

该部分是基于casbot自有的AMP框架训练部署的操作说明，如采用其他算法框架训练部署，则需基于3.3全身关节控制接口进行二次开发编写推理程序。

3.4.1 强化学习环境搭建

casbot02强化学习环境基于Isaac Gym开发，采用AMP强化学习算法进行训练。具体详情见见casbot 02开源项目 链接 git@10.11.0.5:sh/cerebella/hl_casbot02_opensource.git。

3.4.2 强化学习环境训练

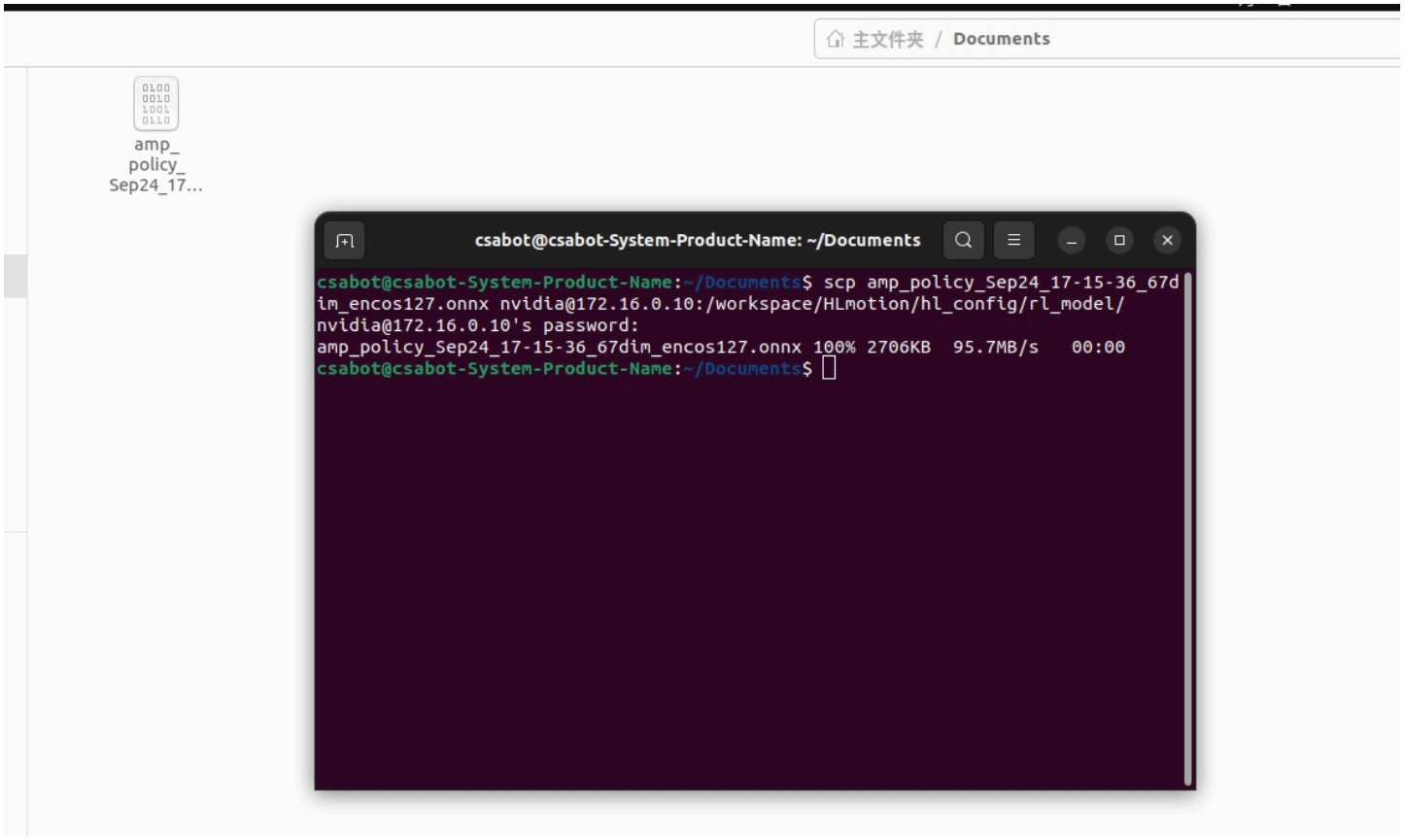
casbot02强化学习环境训练包括train、play、sim2sim，具体详情见见casbot 02开源项目 链接 git@10.11.0.5:sh/cerebella/hl_casbot02_opensource.git。

3.4.3 强化学习模型部署

仿真环境（hl_casbot02_simulation）和casbot02真机的onnx策略文件部署过程是相同的，先将训练好的onnx策略文件拷贝到正确的目录，再修改yaml配置文件。

（1）例如新训练的策略文件为amp_policy_xxxx.onnx；

（2）将amp_policy_xxxx.onnx拷贝到目录/workspace/HLmotion/hl_config/rl_model/目录下，如在仿真环境，则需进行docker操作;



（3）更新yaml参数，打开/workspace/HLmotion/hl_config/rl_config.yaml，将amp_arm节点的model_file_path值改为“/workspace/HLmotion/hl_config/rl_model/amp_policy_xxxx.onnx”；

```yaml
  size:
    actions_size: 12
    observations_size: 47
    history_size: 15
    input_size: 705 # 46*15

amp_arm:
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/02_float_gait_kp20.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/2025-04-10_14-37-32_tabu.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/02_policy_2025-04-28_18-06-10.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/02_policy_2025-04-29_10-51-51.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/amp_policy_May22_11-18-10_.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/amp_policy_Jun09_16-28-51_.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/amp_policy_Jun12_21-22-17_.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/amp_policy_Jun10_17-11-03_.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/amp_policy_Jun16_15-47-16_.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/amp_policy_Jun17_11-01-46_.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/amp_policy_Jun26_11-05-29_yuqi.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/amp_policy_Jul01_17-54-09_yuqi_6k.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/amp_policy_Jul02_21-23-04_yuqi_2500.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/amp_policy0630zp.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/amp_policy_Jul21_19-46-53_px.onnx"
  model_file_path: "/workspace/HLmotion/hl_config/rl_model/amp_policy_Sep24_17-15-36_67dim_encos127.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/amp_policy_Sep24_17-15-36_67dim_encos.onnx
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/amp_policy_Jun18_17-24-15_.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/amp_policy_Jun19_11-13-45_.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/amp_policy_Jun19_12-31-59_ankle_pitch_kp_80.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/amp_policy_Jun19_17-39-59_.onnx"
  # model_file_path: "/workspace/HLmotion/hl_config/rl_model/amp_policy_Jun19_17-39-59_.onnx"
```

操作视频：

